

CONVERSIONI ISTRUZIONI RISC-V

PROVA SCRITTA 26 GIUGNO 2020 - SLIDES

a) $0000\ 0000\ 0010\ 0011\ 0111\ 0011\ 1011\ 0011$
 $RS_2\ RS_1\ RD$
 AND x7, x6, x2 ✓

b) BNE x20, x10, 0xFFC $0xFFC = 1111\ 1111\ 1100$
 $1111\ 1111\ 01010\ 11101\ 001\ 11001\ 1100011$
 $0x\ F\ E\ A\ E\ 0\ C\ E\ 3\ \checkmark$

PROVA SCRITTA 10 LUGLIO 2020 - SLIDES

a) $0000\ 0001\ 1010\ 0010\ 1100\ 0101\ 1011\ 0011$
 $RS_2\ RS_1\ RD$
 XOR x11, x5, x26 ✓

b) BNE x10, x11, 0xC $0xC = 0000\ 0000\ 1100$
 $0000\ 0000\ 01011\ 01010\ 001\ 11000\ 1100011$
 $0x\ 0\ 0\ B\ 5\ 1\ C\ 6\ 3\ \checkmark$

PROVA SCRITTA 25 SETTEMBRE 2020 - SLIDES

a) $0xFEAS9CE3$
 $1111\ 1110\ 1010\ 0101\ 1001\ 1100\ 1110\ 0011$
 $RS_2\ RS_1\ RD$
 BNE x11, x10, 111111111100 = 0xFFC
 BNE x11, x10, 0xFFC ✓

b) XOR x12, x5, x6
 $0000\ 0000\ 00110\ 00101\ 100\ 01100\ 011\ 0011$
 $0x\ 0\ 0\ 6\ 2\ C\ 6\ 3\ 3\ \checkmark$

PROVA SCRITTA 9 GIUGNO 2021 - SLIDES

a) BLT x2, x3, 0x7F0 $0x7F0 = 0111\ 1111\ 0000$
 $0111\ 1111\ 00011\ 00010\ 100\ 00001\ 1100011$
 $0x\ 7\ E\ 3\ 1\ 4\ 0\ E\ 3\ \checkmark$

b) $0x00450513$
 $0000\ 0000\ 0100\ 0101\ 0000\ 0101\ 0001\ 0011$
 $RS_1\ RD$
 ADDI x10, x10, 4 ✓

PROVA SCRITTA 25 GIUGNO 2021 - SLIDES

a) ADDI x10, x10, 4
 $0000\ 0000\ 0100\ 01010\ 000\ 01010\ 001\ 0011$
 $0x\ 0\ 0\ 4\ 5\ 0\ 5\ 1\ 3\ \checkmark$

b) $0x7E3140E3$
 $0111\ 1111\ 00011\ 0001\ 0100\ 0000\ 1110\ 0011$
 $imm\ RS_2\ RS_1\ imm$
 BLT x2, x3, 011111110000 = 0x7F0
 BLT x2, x3, 0x7F0 ✓

PRIMO APPELLO 16 LUGLIO 2021 - ISLFOES

- $0x7F802403$
- $0111\ 1111\ 1000\ \overset{RS_1}{|}0000\ \overset{R_0}{|}010\ |0100\ |0000\ 0011$
- $LW\ x8, 2040(x0)$
- $(1740022003)_8$
- $exp = 11111111$, $MANTISSA \neq 0 \Rightarrow NaN$
- IMPOSSIBILE CODIFICARE IL NUMERO IN BCD PER VIA DI $(1111)_2 = (F)_{16}$.

SALTO MASSIMO E MINIMO ISTRUZIONI BRANCH (SB-TYPE)

IL PROCESSORE CALCOLA L'INDIRIZZO DI SALTO USANDO LA FORMULA:

$$\text{TARGET ADDRESS} = \text{PC.BRANCH} + (\text{IMMEDIATE} \times 2)$$

LE ISTRUZIONI DI TIPO BRANCH SONO CODIFICATE IN FORMATO SB-TYPE, CON UN IMMEDIATO A 12 BIT CON SEGNO. TUTTAVIA, PER IL SALTO EFFETTIVO, L'IMMEDIATO VIENE SHIFTATO A SINISTRA DI 1 (CIOÈ MOLTIPLICATO PER 2), POICHÈ LE ISTRUZIONI SONO ALLINEATE A 4 BYTE E QUINDI I SALTI SONO SEMPRE MULTIPLI DI 2. (QUESTO PER MANTENERE I SALTI COMPATIBILI CON ISTRUZIONI CODIFICATE SU 2 BYTE).

IL VALORE EFFETTIVO DEL SALTO È QUINDI RAPPRESENTATO SU 13 BIT CON SEGNO.

SALTO MASSIMO IN AVANTI (POSITIVO)

1. IL VALORE MASSIMO RAPPRESENTABILE CON 12 BIT SIGNED È:

$$0111\ 1111\ 1111 \rightarrow 0x7FF \rightarrow 2047$$

2. VIENE POI SHIFTATO A SINISTRA DI 1 BIT:

$$0\ 1111\ 1111\ 1110 \rightarrow 0xFFE \rightarrow 4094\ \text{BYTE (13 BIT)}$$

QUINDI IL SALTO MASSIMO IN AVANTI DI UNA ISTRUZIONE SB-TYPE È DI:

$$4094\ \text{BYTE (PC. + 0xFFE)}$$

SALTO MASSIMO ALL'INDIETRO (NEGATIVO)

1. IL VALORE MINIMO RAPPRESENTABILE CON 12 BIT È:

$$1000\ 0000\ 0000 \rightarrow 0x800 \rightarrow -2048$$

2. SHIFTANDO A SINISTRA DI 1 BIT:

$$1\ 0000\ 0000\ 0000 \rightarrow 0x1000 \rightarrow -4096\ \text{BYTE (13 BIT)}$$

QUINDI IL SALTO MASSIMO ALL'INDIETRO DI UNA ISTRUZIONE SB-TYPE È DI:

$$-4096\ \text{BYTE (P.C. - 0x1000)}$$

PROVA SCRITTA 25 SETTEMBRE 2020 - SLIDES

QUAL È L'INDIRIZZO PIÙ GRANDE CHE PUÒ ESSERE RAGGIUNTO TRAMITE L'ISTRUZIONE BEQ, SOPPONENDO CHE IL P.C. CONTENGA IL VALORE 0x2000 0000:

$$0x2000\ 0000 + 0xFFE =$$

$$0x2000\ 0FFE$$

✓

SALTO MASSIMO IN AVANTI E ALL'INDIETRO SAL (US-TYPE)

LE ISTRUZIONI SAL USANO IL FORMATO US-TYPE, CHE CONSENTE UN SALTO PIÙ AMPIO RISPETTO ALLE ISTRUZIONI BRANCH (SB-TYPE).

L'IMMEDIATO È LUNGO 20 BIT, E CON LO STESSO PRINCIPIO VIENE SHIFTATO A SINISTRA DI 1 BIT (CIOÈ MOLTIPLICATO PER 2), QUINDI IL SALTO EFFETTIVO È SO 21 BIT CON SEGNO.

LA FORMULA PER CALCOLARE L'INDIRIZZO DI SALTO RIMANE:

$$\text{TARGET ADDRESS} = \text{PC SAL} + (\text{IMMEDIATE} \times 2)$$

SALTO MASSIMO IN AVANTI (POSITIVO)

1. VALORE MASSIMO 20 BIT SIGNED:

$$0111\ 1111\ 1111\ 1111\ 1111 \rightarrow 0x7FFFF \rightarrow 524.287$$

2. SHIFT A SINISTRA DI 1 BIT:

$$01111\ 1111\ 1111\ 1111\ 1110 \rightarrow 0xFFFFE \rightarrow 1.048.574$$

QUINDI IL SALTO IN AVANTI MASSIMO DI UNA SAL (JUMP AND LINK) È DI:

$$1.048.574 \text{ BYTE (PC} + 0xFFFFE)$$

SALTO MASSIMO ALL'INDIETRO (NEGATIVO)

1. VALORE MINIMO 20 BIT SIGNED:

$$1000\ 0000\ 0000\ 0000\ 0000 \rightarrow 0x80000 \rightarrow -524.228$$

2. SHIFT A SINISTRA DI 1 BIT:

$$10000\ 0000\ 0000\ 0000\ 0000 \rightarrow 0x100000 \rightarrow -1.048.576$$

QUINDI IL SALTO ALL'INDIETRO MASSIMO DI UNA SAL (JUMP AND LINK) È DI:

$$-1.048.576 \text{ BYTE (PC} - 0x100000)$$

SECONDO PARZIALE 18 MAGGIO 2021 - ES 1

QUAL È L'INDIRIZZO PIÙ GRANDE CHE PUÒ ESSERE RAGGIUNTO TRAMITE L'ISTRUZIONE SAL, SUPPONENDO CHE IL P.C. CONTENGA IL VALORE 0x0040 0000.

$$0x0040\ 0000 + 0x000F\ FFFE$$

$$0x004F\ FFFE$$



CONVERSIONI ISTRUZIONI RISC-V

3

PRIMO APPELLO 14 GIUGNO 2022 - ES 1

- 1) 0x0014 0x00052483 LOOP: LW x9, 0(x10)
 2) 0x0018 0x00990933 ADD x18, x18, x9
 3) 0x001C 0x00450513 ADDI x10, x10, 4
 4) 0x0020 0x00140413 ADDI x8, x8, 1
 5) 0x0024 0xFEB448E3 BLT x8, x11, LOOP

1) 0000 0000 0000 | 0101 0100 0100 1000 0011

LW x9, 0(x10)

2) 0000 0000 1001 | 1001 0000 1001 0110 0111

ADD x18, x18, x9

3) 0000 0000 0100 | 0101 0000 0101 0001 0011

ADDI x10, x10, 4

4) 0000 0000 0001 | 0100 0000 0100 0001 0011

ADDI x8, x8, 1

5) 1111 1110 1011 | 0100 0100 1000 1110 0011

BLT x8, x11, 111111111000 = 0xFF8 (12 BIT) << 1

SHIFTED: (0001)111111110000 = 0xFF0 (13 BIT) +

BASEPC: (0000)000000100100 = 0x024 (13 BIT) =

(0010)000000010100 = 0x014 (13 BIT)

REGOLA RISOLUTIVA

IL PROCESSORE CALCOLA L'INDIRIZZO DI SALTO USANDO LA FORMULA:

$$\text{TARGET ADDRESS} = \text{PROGRAM COUNT} + (\text{IMMEDIATE} \times 2)$$

LE ISTRUZIONI DI TIPO SB-TYPE HANNO UN IMMEDIATO A 12 BIT CON SEGNO.

PER IL SALTO EFFETTIVO L'IMMEDIATO VIENE SHIFTATO A SINISTRA DI 1 (x2)

OTTENENDO UN NUMERO A 13 BIT SIGNED (CON SEGNO). DOPODI CHE VIENE

SOMMATO ALL'INDIRIZZO DELLA BRANCH OTTENENDO COME RISULTATO L'INDIRIZZO A

13 BIT DELLA DESTINAZIONE DELLA BRANCH.

SECONDO PARZIALE 18 MAGGIO 2021 - ES 7

1) 0x44 0x0005 2483 LOOP: LD x9, 0(x10)
 2) 0x48 0x0094 0433 ADD x8, x8, x9
 3) 0x4C 0x0045 0513 ADDI x10, x10, 4
 4) 0x50 0xFE35 4AE3 BLT x10, x10, LOOP

1) 0000 0000 0000 | 0101 0010 | 0100 1000 0011

LD x9, 0(x10)

2) 0000 0000 1001 | 0100 0000 | 0100 0011 0011

ADD x8, x8, x9

3) 0000 0000 0100 | 0101 0000 | 0101 0001 0011

ADDI x10, x10, 4

4) 1111 1111 0101 | 0101 0100 | 1010 1110 0011

BLT x10, x11, 111111111010 = 0xFFA < 1 (12 BIT)

0001 111111110100 = 0x1FF4 + (13 BIT)

0000 0000 0101 0000 = 0x0050 = (13 BIT)

0010 0000 0100 0100 = 0x044 (13 BIT)

LI x8, 0

// REGISTER THAT'LL CONTAIN ELEMENTS SUM

SLLI x11, x11, 2

// CALCULATING TOTAL SIZE OF A (INTEGERS)

ADD x11, x11, x10

// CALCULATING LAST ADDRESS OF A.

LOOP: LD x9, 0(x10)

// GETTING CURRENT ELEMENT VALUE THROUGH POINTER

ADD x8, x8, x9

// ADD CURRENT VALUE TO THE SUM.

ADDI x10, x10, 4

// INCREMENT POINTER. (4 BYTES, INT).

BLT x10, x11, LOOP

// LOOPING UNTIL POINTER REACHES LAST ADDR.

Somma tutti gli elementi contenuti nell'array A (x10) tramite l'uso di un puntatore (x10).

Sì, è necessario allocare lo stack perché utilizziamo x8 e x9 (S0, S1):

ADDI SP, SP, -16

...

SD x8, 0(SP)

LD x8, 0(SP)

SD x9, 8(SP)

LD x9, 8(SP)

...

ADDI SP, SP, 16

CONVERSIONI ISTRUZIONI RISC-V

TERZO APPELLO 22 LUGLIO 2022 - ES 1 (CORRETTO)

- 1) 0x4000 0x0004 0A63 L1: BEQ x8, x0, L2
 2) 0x4004 0x00A9 04B3 ADD x9, x18, x10
 3) 0x4008 0x00B4 8023 SB x11, 0(x9)
 4) 0x400C 0x0019 0913 ADDI x18, x18, 1
 5) 0x4010 0xFF1F F06F SAL x0, L1
 6) 0x4014 0x0000 8067 L2: SALR x0, 0(x1)

1) $\begin{array}{|c|c|c|c|c|c|c|c|} \hline \text{imm} & & \text{RS}_2 & & \text{RS}_1 & & \text{imm} & \\ \hline 0000 & 0000 & 0000 & 0100 & 0000 & 1010 & 0110 & 0011 \\ \hline \end{array}$
 BEQ x8, x0, 000000001010 = 0x00A < 1

$\begin{array}{|c|c|c|c|c|c|c|c|} \hline 0000 & 0000 & 0001 & 0100 & & & & \\ \hline 0100 & 0000 & 0000 & 0000 & & & & \\ \hline 0100 & 0000 & 0001 & 0100 & & & & \\ \hline \end{array}$
 = 0x014 +
 = 0x4000 =
 = 0x4014

2) $\begin{array}{|c|c|c|c|c|c|c|c|} \hline & & \text{RS}_2 & & \text{RS}_1 & & \text{RD} & \\ \hline 0000 & 0000 & 1010 & 1001 & 0000 & 0100 & 1011 & 0011 \\ \hline \end{array}$
 ADD x9, x18, x10

3) $\begin{array}{|c|c|c|c|c|c|c|c|} \hline & & \text{RS}_2 & & \text{RS}_1 & & & \\ \hline 0000 & 0000 & 1011 & 0100 & 1000 & 0000 & 0010 & 0011 \\ \hline \end{array}$
 SB x11, 0(x9)

4) $\begin{array}{|c|c|c|c|c|c|c|c|} \hline & & & & \text{RS}_1 & & \text{RD} & \\ \hline 0000 & 0000 & 0001 & 1001 & 0000 & 1001 & 0001 & 0011 \\ \hline \end{array}$
 ADDI x18, x18, 1

5) $\begin{array}{|c|c|c|c|c|c|c|c|} \hline & & & & & & \text{RS}_1 & \\ \hline 1111 & 1111 & 0001 & 1111 & 1111 & 0000 & 0110 & 1111 \\ \hline \end{array}$
 SAL x0, 1111 1111 1111 1111 1000 < 1

$\begin{array}{|c|c|c|c|c|c|c|c|} \hline 0001 & 1111 & 1111 & 1111 & 1111 & 0000 & & \\ \hline 0000 & 0000 & 0100 & 0000 & 0001 & 0000 & & \\ \hline 0010 & 0000 & 0100 & 0000 & 0000 & 0000 & & \\ \hline \end{array}$
 = 0x1FFFFFF0 +
 = 0x4010 =
 = 0x4000

6) $\begin{array}{|c|c|c|c|c|c|c|c|} \hline & & & & \text{RS}_1 & & \text{RD} & \\ \hline 0000 & 0000 & 0000 & 0000 & 1000 & 0000 & 0110 & 0111 \\ \hline \end{array}$
 SALR x0, 0(x1)

RICORDA

- LE STORE SEGUONO LA SINTASSI: <STORE> RS₂, OFFSET(RS₁)

ESAME 19 GEHHAIO 2024 - ES 2 L1: BEQ x6, x0, L2

- 1) 0x00400000 0x0003 0863 ADDI x6, x6, 4.095
- 2) 0x00400004 0xFFFB 0313 ADDI x5, x5, 2
- 3) 0x00400008 0x0022 8293 SAR x0, L1
- 4) 0x0040000C 0xFF5F F06F L2: ...

1)
$$\begin{array}{|c|c|c|c|c|c|c|c|c|c|} \hline \text{RS}_2 & & & & \text{RS}_1 & & & & & \\ \hline 0000 & 0000 & 0000 & 0011 & 0000 & 1000 & 0110 & 0011 & & \\ \hline \end{array}$$

BEQ x6, x0, 000000001000 = 0x008 << 1

$$000000010000 = 0x010 +$$

$$0x0040/000000000000 = 0x00400000 =$$

$$0x0040/000000010000 = 0x00400010$$

2)
$$\begin{array}{|c|c|c|c|c|c|c|c|c|c|} \hline & & & \text{RS}_1 & & \text{RD} & & & & \\ \hline 1111 & 1111 & 1111 & 0011 & 0000 & 0011 & 0001 & 0011 & & \\ \hline \end{array}$$

ADDI x6, x6, 4.095 $(2^{11} - 1)$

3)
$$\begin{array}{|c|c|c|c|c|c|c|c|c|c|} \hline & & & \text{RS}_1 & & \text{RD} & & & & \\ \hline 0000 & 0000 & 0010 & 0010 & 1000 & 0010 & 1001 & 0011 & & \\ \hline \end{array}$$

ADDI x5, x5, 2

4)
$$\begin{array}{|c|c|c|c|c|c|c|c|c|c|} \hline & & & & & & & & & \\ \hline 1111 & 1111 & 0101 & 1111 & 1111 & 0000 & 0110 & 1111 & & \\ \hline \end{array}$$

SAR x0, 11111111111111111010 = 0xFFFFFA << 1

$$11111111111111111100 = 0x1FFFFFF4 +$$

$$00000000000000001100 = 0x0040000C =$$

$$0x00/010000000000000000000000 = 0x00400000$$

CONVERSIONI ISTRUZIONI RISC-V

5

SECONDO APPELLO 4 LUGLIO 2022 - ES 4

L1: ADD S1, S2, A0

SB A1, 0(S1)

BEQ S0, ZERO, L2

ADDI S2, S2, 1

SAL ZERO, L1

L2: SALR ZERO, 0(RA)

L2 → 3 AVANTI → $3 \cdot 4 = 12$ (12 BIT)

→ 0000 0000 1100 >> 1

→ 0000 0000 0110 =

$0x006 = (6)_{10}$

L1 → 4 INDIETRO → $-4 \cdot 4 = -16$ (20 BIT)

1111 1111 1111 1110 1111 >> 1

1111 1111 1111 1111 0111 =

1111 1111 1111 1000 =

→ $0xFFFF8 = (-8)_{10}$

1) ADD x9, x18, x10

2) SB x11, 0(x9)

3) BEQ x8, x0, 0x006

4) ADDI x18, x18, 1

5) SAL x0, 0xFFFF8

6) SALR x0, 0(x1)

1) 00000000 01010 10010 000 01001 0110011 0x00A904B3

2) 00000000 01011 01001 000 00000 0100011 0x00B48023

3) 00000000 00000 01000 000 01100 1100011 0x00040663

4) 00000000 00001 10010 000 10010 0010011 0x00190913

5) 11111111 00011 11111111 00000 1101111 0xFF1FF06F

6) 00000000 00000 00001 000 00000 1100111 0x00008067